Day 173: Experiment Tracking

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 6 | Complete study guide

Watch: MLFlow Tutorial Part 1: Experiment Tracking (Anton T. Ruberts) | Machine Learning Experiment Tracking with VS Code, Git, and DVC (DVCorg)

Script: `day173_experiment_tracking.py` · every number in this guide is copied by program from `day173_run_output.txt`, the real output of that script.

A tuning study is dozens of runs, and a month later the questions are always the same: which settings gave that number, on which data, with which code and seed, and can it be reproduced? Today builds a small tracker from scratch (no MLflow) and runs a real 40-configuration study with it on UCI Adult. The study crashed at run 17 on purpose and resumed without redoing 16 finished runs; 5 of 5 logged runs were reproduced bit for bit from their records alone; and the log showed that the winner's score was partly its seed: +0.0039 better than the same settings with ten new seeds. A control showed about half of that (+0.0022) was seed 0 being a good seed for configurations nobody chose.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

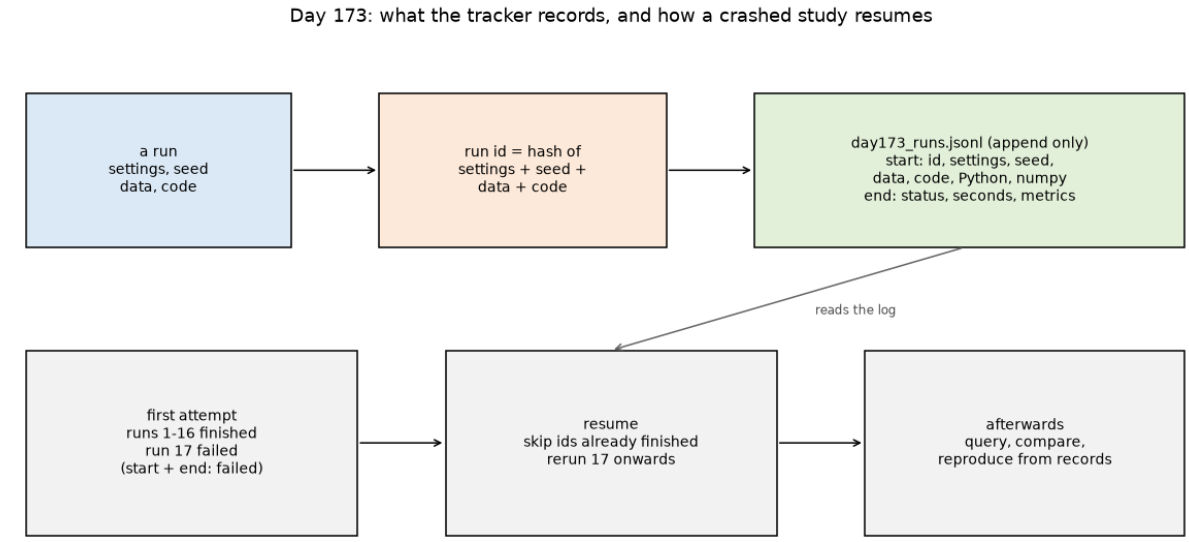
Experiment tracking means writing every run down automatically, at the moment it runs: its **settings** (hyperparameters), its **seed**, a **data version** and a **code version** (here short hashes of the arrays and of the script), the **environment** (Python and numpy versions), and its **results**, including the curve over training. A **run id** is a hash of everything that defines the run, so the same run always gets the same id.

The log here is **JSON Lines**: one JSON record per line, only ever appended. A run writes a **start** record before it trains and an **end** record after, so a crash still leaves a trace. Tools such as MLflow, Weights & Biases and DVC do the same with a database and a web page on top.

REAL-WORLD SCENARIO

A lab notebook. A chemist who writes 'good result, about 80% yield' on a loose sheet cannot repeat it; one who writes the batch of every reagent, the temperature and the time, in a bound book with dated pages, can. Tracking is the bound book, written by the code itself so nothing is forgotten.

SYSTEM DIAGRAM



What a run record holds, and how a crashed study resumes by reading its own log.

Data: Week 24's split of UCI Adult, as on Day 171: 2,000 training rows, a 6,000-row stop set for early stopping, and the 6,032 validation rows, which choose. The test file is not touched. The model is Day 126's network trained by Day 171's `train_es` with Adam and early stopping; 40 configurations of learning rate (log-uniform from 1e-4 to $10^{-1.5}$), patience (1 to 20) and batch size (32 to 256).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A tracked random search that crashes, then resumes

DEFINITION — start here, no prior knowledge assumed

Each run is wrapped in `with tracker.run(settings, seed, data, code) as r:`. Entering the block writes the start record; leaving it, normally or by an exception, writes the end record with the status, the time taken, the per-epoch stop-set loss and the final scores.

Run 17 raises an error on the first attempt. The study is then simply started again: before each run it asks the log whether that exact run id has already finished, and skips it if so.

REAL-WORLD SCENARIO

A long download that resumes from where it broke, instead of starting again from zero.

MATHEMATICS

run id = first 12 hex digits of sha256(JSON of {settings, seed, data hash, code hash}, keys sorted)

data hash = sha256(shape, type and bytes of every array) code hash = sha256(the script's bytes)

IMPLEMENTATION

```
class Run:
    def __init__(self, tracker, params, seed, data, code):
        self.t, self.rec = tracker, dict(id=run_id(params, seed, data, code), params=params,
            seed=seed, data=data, code=code)
        self.history, self.summary = [], {}
    def log(self, step, **metrics):
        self.history.append(dict(step=step, **{k: float(v) for k, v in metrics.items()}))
    def __enter__(self):
        self.t0 = time.time()
        self.t.write(dict(event="start", time=round(self.t0, 3), env=self.t.env, **self.rec))
        return self
    def __exit__(self, kind, err, tb):
        self.t.write(dict(event="end", id=self.rec["id"], status="failed" if kind else "finished",
                          error=repr(err) if kind else None, seconds=round(time.time() - self.t0,
                              3),
                          summary=self.summary, history=self.history))
        return False
```

IMPLEMENTATION

```
class Tracker:
    def __init__(self, path):
        self.path = path
        self.env = dict(python=platform.python_version(), numpy=np.__version__)
    def write(self, record):
        with open(self.path, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, sort_keys=True) + "\n")
    def run(self, params, seed, data, code):
        return Run(self, params, seed, data, code)
    def runs(self):
        out = {}
        if not os.path.exists(self.path):
            return []
        for line in open(self.path, encoding="utf-8"):
            r = json.loads(line)
            if r["event"] == "start":
                out[r["id"]] = dict(r, status="unfinished")
            else:
                out[r["id"]].update(status=r["status"], error=r["error"], seconds=r["seconds"],
                                    summary=r["summary"], history=r["history"])
        return list(out.values())
    def done(self, rid):
        return any(r["id"] == rid and r["status"] == "finished" for r in self.runs())
```

ACTUAL OUTPUT

```
first attempt stopped: simulated crash in run 17
log after the crash: 16 finished, 1 failed (RuntimeError('simulated crash in run 17'))
resumed: 16 runs skipped as already finished, 24 run now; the log holds 40 runs, 40 finished
82 lines in day173_runs.jsonl, 71 KB; data ed1c86d3dc20, code 00a6b58363c8; (12 s)
```

What it means

The crash cost nothing that had finished. After it the log held 16 finished, 1 failed, with the error text recorded. The second call skipped 16 runs and ran 24, ending with all 40 finished. The failed attempt stays in the file; when the log is read back, the latest attempt of an id wins.

The log is small and plain. 82 lines (a start and an end for 40 runs, plus the failed attempt's two), 71 KB, readable in any text editor; most of it is the per-epoch curves.

WHY CHOOSE THIS?

Append-only lines survive crashes: a half-written study is still a valid log, and nothing is ever overwritten.

WHY NOT THIS? (TRADEOFFS)

Reading means scanning the whole file; at tens of thousands of runs a database (what MLflow uses) is faster.

2. Querying the log

DEFINITION — start here, no prior knowledge assumed

Once every run is in the log, questions can be asked after the fact without rerunning anything: the best runs, how each setting relates to the score, or how one slice of the runs behaved.

REAL-WORLD SCENARIO

A spreadsheet of every past sale lets a shop ask new questions about old customers.

IMPLEMENTATION

```
def query(runs, where=lambda r: True, by=None, n=None):
    out = [r for r in runs if r["status"] == "finished" and where(r)]
    if by is not None:
        out.sort(key=lambda r: r["summary"][by])
    return out[:n] if n else out
```

ACTUAL OUTPUT

```
  the five best runs by validation log loss:
    f45132550c3b  lr 1.84e-04  patience  4  batch 128  ->  val 0.3385, best epoch 66, 70 epochs, 0.69 s
    d101e475f6ee  lr 1.77e-04  patience  4  batch 128  ->  val 0.3385, best epoch 66, 70 epochs, 0.67 s
    645d7f93bce5  lr 2.10e-04  patience 15  batch 256  ->  val 0.3385, best epoch 95, 100 epochs, 1.04 s
    d717a098b146  lr 1.45e-04  patience 19  batch 128  ->  val 0.3386, best epoch 88, 100 epochs, 1.27 s
    3bea164b1ee4  lr 1.11e-04  patience  6  batch 128  ->  val 0.3386, best epoch 99, 100 epochs, 1.03 s
  log10 lr  : correlation with validation loss +0.79
  patience  : correlation with validation loss -0.18
  log2 batch: correlation with validation loss -0.38
  runs with lr < 3e-4: 8, mean epochs 73.5, mean val 0.3389; the rest: mean val 0.3444
```

What it means

The learning rate mattered most. Its log correlated +0.79 with the validation loss: smaller rates were better. The eight runs below 3e-4 averaged 0.3389 against 0.3444 for the rest, and ran 73.5 epochs on average, so they were also the slowest.

Patience (-0.18) and batch size (-0.38) mattered less. Larger batches did slightly better; a likely reason, not tested here, is that with at most 100 epochs a larger batch takes fewer steps, which acts like a smaller learning rate.

The top five are nearly one configuration. Learning rates from 1.1e-4 to 2.1e-4, all at 0.3385 or 0.3386. Section 4 asks whether their order means anything.

WHY CHOOSE THIS?

Questions nobody thought of in advance can still be answered from a complete log.

WHY NOT THIS? (TRADEOFFS)

Correlations across 40 runs are descriptive; they do not separate settings that were drawn together.

3. Reproducing runs from their records alone

DEFINITION — start here, no prior knowledge assumed

Every eighth finished run (best first) was retrained using only what its record says: the settings and the seed. The new validation loss is compared with the logged one to the last bit, written with `repr` so no rounding hides a difference.

REAL-WORLD SCENARIO

Following a recipe card written by someone else and getting the same cake.

IMPLEMENTATION

```
for r in picks:
    val, best, ep = train_and_score(D, r["params"], r["seed"])
    ok = val == r["summary"]["val_loss"] and best == r["summary"]["best_epoch"] and ep ==
        r["summary"]["epochs"]
    same += ok
```

ACTUAL OUTPUT

```
  f45132550c3b: logged val 0.33849483922349605, rerun 0.33849483922349605; identical: True
  462b274d7337: logged val 0.3390067135740327, rerun 0.3390067135740327; identical: True
  7607dbc39d9b: logged val 0.3403018890474152, rerun 0.3403018890474152; identical: True
  950ef5a09d15: logged val 0.3423205796216293, rerun 0.3423205796216293; identical: True
  32d57eeecd8d: logged val 0.34997097718587017, rerun 0.34997097718587017; identical: True
  5 of 5 reproduced bit for bit (Python 3.14.2, numpy 2.5.0)
```

What it means

5 of 5 matched bit for bit, for example 0.33849483922349605. The record was enough: the settings and seed fix the weights' starting values and the order of the batches, and the data and code hashes say what else must be the same.

What this does not show. The reruns happened in the same Python process on the same machine. A fresh process, another machine, another numpy or another number of BLAS threads can change the last bits of a sum. Day 174 reruns a whole study from scratch to test that.

4. The seed: how much of the winner's score was luck?

DEFINITION — start here, no prior knowledge assumed

The study ran each configuration once, with seed 0. The top five configurations were rerun with ten new seeds each (1 to 10). As a control, five configurations from the middle of the ranking (ranks 18 to 22), which were not chosen for a good score, got the same ten seeds.

REAL-WORLD SCENARIO

A runner picked for the team after one race, timed again on ten other days.

MATHEMATICS

$$optimism = mean\ over\ seeds\ 1..10\ of\ val\ loss - logged\ seed\text{-}0\ val\ loss$$

$$selection\ effect \approx optimism(top\ 5) - optimism(control)$$

ACTUAL OUTPUT

```
  rank 1 (f45132550c3b): logged 0.3385; new seeds mean 0.3423 (sd 0.0017, 0.3402 to 0.3451)
  rank 2 (d101e475f6ee): logged 0.3385; new seeds mean 0.3423 (sd 0.0017, 0.3402 to 0.3452)
  rank 3 (645d7f93bce5): logged 0.3385; new seeds mean 0.3426 (sd 0.0017, 0.3406 to 0.3453)
  rank 4 (d717a098b146): logged 0.3386; new seeds mean 0.3425 (sd 0.0018, 0.3403 to 0.3454)
  rank 5 (3bea164b1ee4): logged 0.3386; new seeds mean 0.3425 (sd 0.0018, 0.3405 to 0.3454)
  the logged winner's score 0.3385 against its new-seed mean 0.3423: +0.0039
  ranked by new-seed means, the logged ranks 1 to 5 become: [2, 1, 5, 4, 3]
  new-seed mean minus logged score: top 5 [0.0039, 0.0038, 0.004, 0.0039, 0.0039]; control ranks 18-22 [0.0034, 0.0015,
      0.0022, 0.0019, 0.002]
  means: top +0.0039, control +0.0022; seed 0 helped the unselected controls too, and the top 5 gain +0.0017 more from being
      chosen
```

What it means

Every top configuration was worse with new seeds: the winner's 0.3385 became a mean of 0.3423, +0.0039; even its best new seed scored 0.3402.

The control split that into two parts. The unselected configurations also lost +0.0022 on average: seed 0 was simply a good seed for this network on this data. The top five lost +0.0017 more, the part that came from being chosen out of 40 (Day 168's winner's curse).

The ranking among the top five was noise. Ordered by their new-seed means they became [2, 1, 5, 4, 3], and the means differed by less than the seed spread (sd 0.0017). Without the seed in the log, none of this could have been checked.

WHY CHOOSE THIS?

Logging the seed makes a single lucky run visible and testable, and rerunning a few finalists is cheap.

WHY NOT THIS? (TRADEOFFS)

Ten seeds for every configuration would multiply the study's cost by ten; spend them on the finalists.

5. Fingerprints: a silently changed dataset, a changed line of code, and the cost

DEFINITION — start here, no prior knowledge assumed

About 1% of the training rows were quietly dropped and the best configuration was rerun with the same settings and seed. Then one value in the training data was changed by a millionth, and one constant in this script by one character. Finally, the tracker's own work was timed.

REAL-WORLD SCENARIO

A seal on a jar: it does not say what changed inside, only that something did.

IMPLEMENTATION

```
def fingerprint(*arrays):
    h = hashlib.sha256()
    for a in arrays:
        a = np.ascontiguousarray(a); h.update(f"{a.shape}{a.dtype}".encode());
            h.update(a.tobytes())
    return h.hexdigest()[:12]
def file_hash(path):
    return hashlib.sha256(open(path, "rb").read()).hexdigest()[:12]
```

ACTUAL OUTPUT

```
  17 of 2000 training rows dropped: data ed1c86d3dc20 -> 8e22cb683997; same settings and seed, val 0.3385 -> 0.3384; run ids
      differ: True
  one value changed by 1e-6: fingerprint a5ea75426ce5 against ed1c86d3dc20
  this script 00a6b58363c8; with one constant changed 6b7276b0c6a4
  cost: a run took 0.31 s on average; reading and searching the 41-run log took 2.5 ms
```

What it means

The score could not tell the data had changed. Dropping 17 of 2000 rows moved the validation loss from 0.3385 -> 0.3384, a difference anyone would call noise. The data hash changed, so the run got a new id and the two runs can never be confused.

Hashes catch any change. One value changed by 1e-6 gave a5ea75426ce5 instead of ed1c86d3dc20; a one-character edit to the script changed its hash too. A hash says that something changed, not what or whether it matters.

Tracking is almost free here. A run took 0.31 s on average; reading and searching the whole log took 2.5 ms.

WHY CHOOSE THIS?

Hashing data and code turns 'I think it was the same data' into a check.

WHY NOT THIS? (TRADEOFFS)

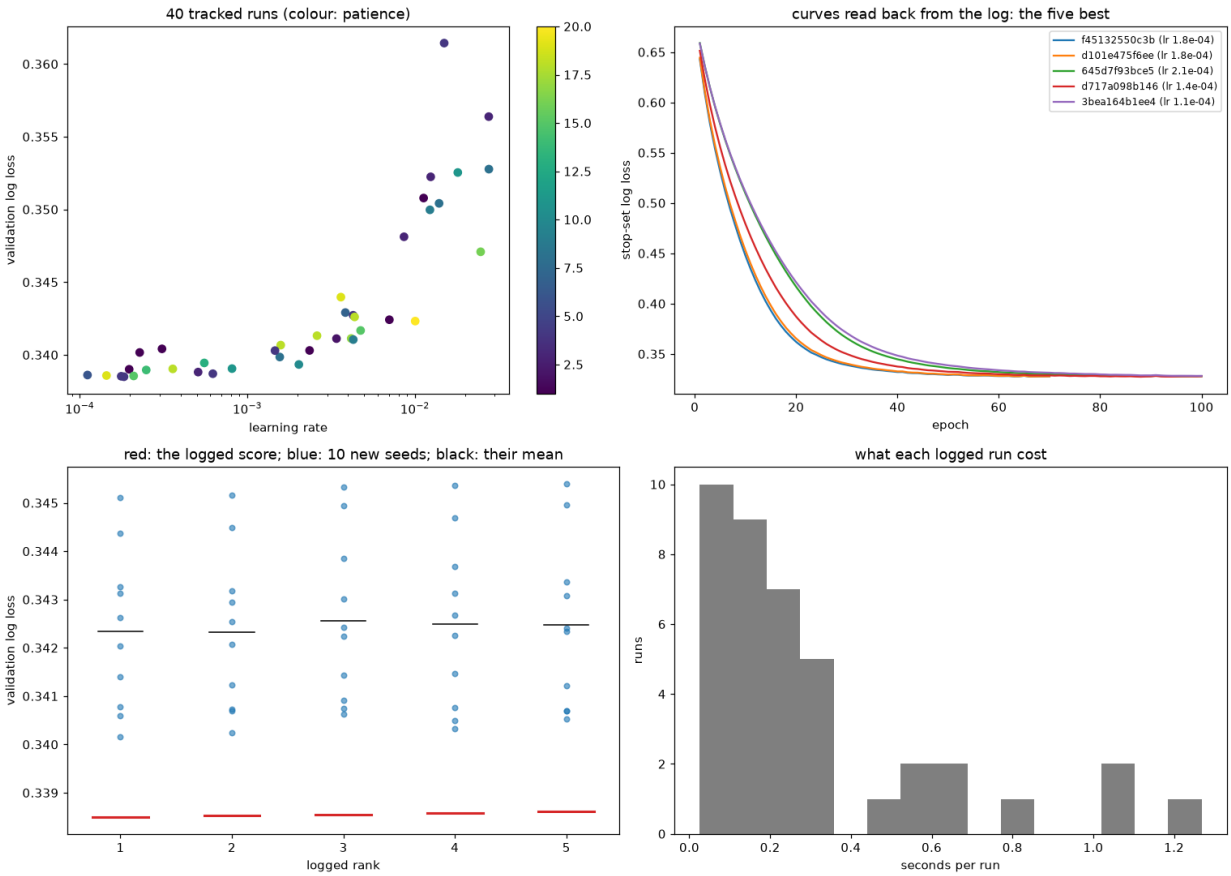
Hashing huge datasets every run is slow; real tools hash files once, or record a dataset version instead.

Today's part for the week's build

`day173_exercise.py` holds `fingerprint`, `file_hash`, `run_id`, `Run`, `Tracker` and `query`, copied unchanged. Its checks run a three-run toy study in a temporary folder, crash it, resume it, and count the log's lines by hand:

ACTUAL OUTPUT

```
fingerprint: equal copies match True, one value changed False, reshaped False, float64 False (by
    hand True, False, False, False)
run_id ignores dictionary order: True, but not the seed: False (by hand True, False)
after the crash: statuses ['finished', 'failed'], lines 4 (by hand ['finished', 'failed']: lr 0.4
    never started; 4 lines, a start and an end each)
after resuming: statuses ['finished', 'finished', 'finished'], lines 8 (by hand three finished; 4
    + 2 for lr 0.2 again + 2 for lr 0.4 = 8)
query by score: lr [0.2, 0.1, 0.4], scores [0.0, 0.01, 0.04] (by hand 0.2 first with 0, then 0.1
    with 0.01, then 0.4 with 0.04)
```



Top left: the 40 runs by learning rate, coloured by patience. Top right: the five best runs' stop-set curves, read back from the log. Bottom left: the top five's logged seed-0 scores (red) against ten new seeds (blue) and their mean (black). Bottom right: how long each run took.

Interview preparation — technical questions

Q: What should a run record hold?

A: Settings, seed, data version, code version, environment, status, time, final metrics and the training curve.

Q: How do you make a tuning study resumable?

A: Give each run a deterministic id and skip ids the log already shows as finished; here 16 runs were skipped after a crash.

Q: How do you know two runs used the same data?

A: Compare a hash of the data; dropping 17 rows changed the hash but moved the score by only 0.0001.

Q: Why log a start record before training?

A: So a crash or a killed job still leaves a record of what was running.

Q: Your best run beat the rest by a hair. What do you check?

A: Rerun the finalists with new seeds; here the top five's order became [2, 1, 5, 4, 3].

Interview preparation — theoretical questions

Q: Why append rather than update records?

A: Nothing is ever lost or half-overwritten; the history of failed attempts stays visible.

Q: What does a hash prove and what does it not?

A: Equal hashes mean identical bytes; different hashes say something changed, not how much it matters.

Q: Why is a reproduced number in the same process weak evidence?

A: Threads, library versions and hardware can change floating-point sums; reproducibility must be tested in a fresh run.

Q: Why compare against a control when measuring the winner's curse?

A: Part of the drop can be a shared effect, like a good seed; here it was about half.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A result is only as trustworthy as its record. Written automatically, the record let a crashed study resume, answered questions nobody planned, reproduced runs bit for bit, and showed that the winner owed part of its score to its seed and part to being chosen.

- Crash at run 17: 16 finished runs skipped on resume, none redone.
- 5 of 5 runs reproduced from their records alone, in the same process.
- The winner's score was +0.0039 optimistic; +0.0022 of that was a good seed, shared by unchosen configurations.
- 17 dropped rows moved the score by 0.0001 but changed the data hash.

Suggested watch order, if starting from zero

- 1. MLFlow Tutorial Part 1: Experiment Tracking (Anton T. Ruberts) — logging parameters, metrics and models with MLflow.
- 2. Machine Learning Experiment Tracking with VS Code, Git, and DVC (DVCorg) — a history of experiment tracking, and tracking experiments with Git and DVC.

Day 174 preview

- **Reproducibility.** One tracked tuning study built from all six of this week's parts, run twice from scratch to bit-identical results, with seeds controlled and the environment pinned.